

```
In [ ]: import marimo as mo
```

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção " `box` — grupo genérico de células"),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção " `cube` e `path` "),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção

"Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

`box` — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor` ou `None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

`cube` e `path` — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve

ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box`, `cube`, `path`, ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \dots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantires que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);

- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);
- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui`, gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box` , sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube` .
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box` , de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontres.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

Resolução de Sudoku Genérica como Problema de Satisfação de Restrições (CSP)

Trabalho prático nº1

Realizado por: Catarina Rodrigues (a111491) e Francisca Cardoso (a112105)

Licenciatura em Ciências da Computação

Descrição do problema

O Sudoku é formulado matematicamente como um **Problema de Satisfação de Restrições** (CSP) generalizado para grelhas de dimensão $n^2 \times n^2$, onde $n \geq 2$:

Ao olhar para um Sudoku tradicional, parece que temos de programar muitas regras diferentes: uma para linhas, outra para colunas e outras para os blocos 3x3. Contudo, a regra é rigorosamente a mesma em todos os casos: **"nenhum número se pode repetir neste conjunto de células"**.

A única coisa que muda entre uma linha, uma coluna ou um bloco é a sua geometria (ou seja, quais são as coordenadas que lá pertencem). Por isso, em vez de inventar uma estrutura complicada para cada uma, criamos a classe genérica `Box` :

- Uma `Box` é apenas uma "caixa" ou "saco" de células. Não sabe se é uma linha, coluna ou um quadrado.
- A única responsabilidade da `Box` é guardar células, verificar se as coordenadas estão dentro dos limites da grelha e dizer quais os valores que já estão fixos.

Escolha do dicionário

O enunciado deixava à nossa escolha como guardar as células dentro de cada grupo. Decidimos usar um **dicionário de Python** (`(linha, coluna): valor`) pelos seguintes motivos práticos:

1. **Poupa muita memória:** Se usasse uma matriz 9x9 completa para representar uma linha de 9 números, teria 72 posições vazias desnecessárias. Com o dicionário, guardamos apenas as 9 células que realmente importam.
2. **É mais rápido:** No Python, verificar se uma coordenada está dentro de um dicionário (`(r, c) in box.cells`) é praticamente instantâneo.
3. **Aceita qualquer formato:** Um dicionário não se importa com a forma geométrica. Consegue guardar uma linha reta, um quadrado ou uma diagonal sem ter de mudar uma única linha de código.

```
In [ ]: import random
        from ortools.sat.python import cp_model
```

```
In [ ]: #Célula: Implementação do Requisito R1 - Classe Genérica 'Box'

class Box:
    """Representa qualquer grupo de células sujeito à restrição 'todos
    diferentes'.

    Armazena o mapeamento (i, j) -> valor (ou None se a célula estiver
    livre).
    Valida limites de coordenadas e valores permitidos na grelha n^2 x n^2.
    """

    def __init__(self, n=3, initial_cells=None):
        self.n = n
        self.size = n * n # Dimensão da grelha: n^2 (ex: 9 para n=3, 4 para
n=2)
        self.cells = {}
```

```

        if initial_cells:
            for (i, j), val in initial_cells.items():
                self.add(i, j, val)

    def add(self, i, j, val=None):
        """Adiciona a célula (i, j) ao grupo, com valor opcional val.

        Rejeita e levanta ValueError se:
        - Coordenadas fora de [0, n^2 -1]
        - Valor fora de [1, n^2]
        """

        # Validar coordenadas
        if not (0 <= i < self.size and 0 <= j < self.size):
            raise ValueError(
                f"Coordenadas ({i}, {j}) fora dos limites da grelha [0, {self.size -1}]."
            )

        # Validar valor numérico (se fornecido)
        if val is not None:
            if not isinstance(val, int) or not (1 <= val <= self.size):
                raise ValueError(
                    f"Valor {val} inválido. Deve ser um número inteiro no intervalo [1, {self.size}]."
                )
            self.cells[(i, j)] = val

    def to_matrix(self):
        """Devolve a representação do grupo em matriz n^2 x n^2.

        Preenche com 0 as células não pertencentes ou sem valor fixo, e com
        o respetivo inteiro nas células com valor fixado.
        """
        matrix = [[0 for _ in range(self.size)] for _ in range(self.size)]
        for (i, j), val in self.cells.items():
            if val is not None:
                matrix[i][j] = val
        return matrix

    def __repr__(self):
        return f"Box(n={self.n}, celulas={len(self.cells)})"

```

In []: # Célula: Validação automática do Requisito R1

```

def test_box():
    b = Box(n=3)

    # Adicionar células válidas
    b.add(0, 0, val=5)
    b.add(1, 2)
    assert (0, 0) in b.cells and b.cells[(0, 0)] == 5
    assert (1, 2) in b.cells and b.cells[(1, 2)] is None

    # Testar matriz gerada

```

```

m = b.to_matrix()
assert m[0][0] == 5
assert m[1][2] == 0
assert len(m) == 9 and len(m[0]) == 9

# Testar se rejeita coordenadas inválidas (ex: linha 9 fora de 0..8)
try:
    b.add(9, 0)
    assert False, "Devia ter rejeitado coordenada fora de limites"
except ValueError:
    pass

# Testar se rejeita valores inválidos (ex: 10 para n=3)
try:
    b.add(0, 1, val=10)
    assert False, "Devia ter rejeitado valor fora de [1, n^2]"
except ValueError:
    pass

return "Requisito R1 validado com sucesso"

test_box()

```

'Requisito R1 validado com sucesso'

Especializações Geométricas: Cube e Path (R2 e R3)

Com a classe base `Box` pronta, criamos duas subclasses recorrendo a herança (`Cube(Box)` e `Path(Box)`):

- **Não é necessário reinventar:** Como herdamos de `Box`, não precisamos de voltar a escrever o código que valida limites ou que exporta a matriz. Apenas precisamos de definir em cada classe quais as coordenadas que pertencem à sua forma.
- **Cube (Blocos $n \times n$):** Recebe o índice do bloco (por exemplo, bloco `(0, 0)`) e calcula automaticamente o ponto onde ele começa (`bi * n, bj * n`), preenchendo as $n \times n$ células que formam esse quadrado.
- **Path (Linhas e Colunas retas):** Recebe um ponto inicial e final. Valida se o caminho é mesmo reto (horizontal ou vertical) e descobre automaticamente o sentido do percurso (com `step = 1` ou `-1`), garantindo que funciona mesmo que passe as coordenadas do fim para o princípio.

In []: `# Implementação dos requisitos R2 (Cube) e R3 (Path)`

```

class Cube(Box):
    """Especialização de Box que representa um bloco n x n.
    O canto superior esquerdo é a célula (bi * n, bj * n), onde (bi, bj)
    são os índices do bloco com 0 <= bi, bj < n.
    """

```

```

def __init__(self, bi, bj, n=3):
    super().__init__(n=n)

    # Validação dos índices de bloco
    if not (0 <= bi < n and 0 <= bj < n):
        raise ValueError(
            f"Índices de bloco ({bi}, {bj}) fora dos limites permitidos
[0, {n - 1}]."
        )

    self.bi = bi
    self.bj = bj

    # Coordenadas do canto superior esquerdo
    start_row = bi * n
    start_col = bj * n

    # Preenche automaticamente todas as n x n células do bloco
    for r in range(n):
        for c in range(n):
            self.add(start_row + r, start_col + c)

class Path(Box):
    """Especialização de Box que representa um troço reto (R3).
    Gera a sequência contínua entre as coordenadas 'start' (r1, c1) e 'end'
    (r2, c2), inclusive.
    Funciona tanto no sentido direto como no sentido inverso, e apenas para
    troços puramente horizontais ou verticais.
    """

    def __init__(self, start, end, n=3):
        super().__init__(n=n)
        r1, c1 = start
        r2, c2 = end

        # Validação de alinhamento reto (mesma linha ou mesma coluna)
        if r1 != r2 and c1 != c2:
            raise ValueError(
                f"O troço entre {start} e {end} não é reto (não é
horizontal nem vertical)."
            )

        # Construção do troço horizontal (mesma linha)
        if r1 == r2:
            step = 1 if c2 >= c1 else -1
            for c in range(c1, c2 + step, step):
                self.add(r1, c)

        # Construção do troço vertical (mesma coluna)
        else:
            step = 1 if r2 >= r1 else -1
            for r in range(r1, r2 + step, step):
                self.add(r, c1)

```

```
In [ ]: # Validação automática dos requisitos R2 e R3
```

```

def test_cube_and_path():
    # --- Testes de R2: Cube ---
    # Para n=3, o bloco central (1, 1) tem canto superior esquerdo em (3,
3)
    bloco_central = Cube(1, 1, n=3)
    assert len(bloco_central.cells) == 9
    assert (3, 3) in bloco_central.cells
    assert (5, 5) in bloco_central.cells
    assert (2, 2) not in bloco_central.cells

    # Rejeição de bloco inválido (ex: bloco 3 para n=3 só vai até 2)
    try:
        Cube(3, 0, n=3)
        assert False, "Deveria ter rejeitado índice de bloco fora de [0, n-
1]"
    except ValueError:
        pass

    # --- Teste de R3: Path ---
    # Testar linha horizontal no sentido direto e inverso
    linha_dir = Path((0, 0), (0, 8), n=3)
    linha_inv = Path((0, 8), (0, 0), n=3)
    assert len(linha_dir.cells) == 9
    assert len(linha_inv.cells) == 9
    assert linha_dir.cells.keys() == linha_inv.cells.keys()

    # Testar coluna vertical no sentido direto e inverso
    col_dir = Path((0, 2), (8, 2), n=3)
    col_inv = Path((8, 2), (0, 2), n=3)
    assert len(col_dir.cells) == 9
    assert (4, 2) in col_dir.cells
    assert col_dir.cells.keys() == col_inv.cells.keys()

    # Rejeição de troço diagonal (não reto)
    try:
        Path((0, 0), (4, 4), n=3)
        assert False, "Deveria ter rejeitado troço diagonal"
    except ValueError:
        pass

    return "Requisitos R2 (Cube) e R3 (Path) validados com sucesso!"

test_cube_and_path()

```

'Requisitos R2 (Cube) e R3 (Path) validados com sucesso!'

Geração aleatória de pistas iniciais (R4)

Para sortear as pistas iniciais do puzzle:

- **Não precisamos de criar classes novas:** Uma lista de pistas é apenas mais um grupo de células fixas. Por isso, a função devolve diretamente uma instância simples de `Box`.

- **Porquê `random.sample`** : Se sorteássemos coordenadas uma a uma com ciclos normais, o computador podia sortear a mesma célula duas vezes por azar. Ao usar `random.sample` , garantimos que o Python escolhe exatamente k posições distintas na grelha sem qualquer repetição.

```
In [ ]: # Implementação do requisito R4 - Geração aleatória de pistas

def generate_random_clues(n=3, k=None):
    """Gera um grupo (Box) com k células aleatórias preenchidas com valores em [1, n^2].
    - n: fator de dimensão da grelha (grelha n^2 x n^2).
    - k: número de pistas a gerar. Por omissão, assume k = n (ordem de grandeza de n).
    Retorna uma instância da classe base Box contendo apenas as células fixas.
    """
    size = n * n

    # Se k não for especificado, assume k = n conforme sugerido no enunciado
    if k is None:
        k = n

    # Validação do número de pistas
    if k < 0 or k > size * size:
        raise ValueError(
            f"Número de pistas k={k} inválido para uma grelha de dimensão {size}x{size}."
        )

    # Cria a caixa genérica para acolher as pistas
    clues_box = Box(n=n)

    # Gera todas as coordenadas possíveis (linha, coluna) da grelha
    all_coords = [(r, c) for r in range(size) for c in range(size)]

    # Seleciona k coordenadas distintas aleatoriamente (sem repetição de posição)
    chosen_coords = random.sample(all_coords, k)

    # Atribui um valor aleatório em [1, n^2] a cada célula escolhida
    for r, c in chosen_coords:
        val = random.randint(1, size)
        clues_box.add(r, c, val=val)

    return clues_box
```

```
In [ ]: # Validação automática do requisito R4

def test_random_clues():
    n = 3
    size = n * n

    # Testar valor por omissão (k deve ser n = 3)
```

```

default_box = generate_random_clues(n=n)
assert isinstance(default_box, Box), "0 retorno tem de ser um objeto
Box"
assert (
    len(default_box.cells) == n
), f"Deveria ter gerado {n} pistas por omissão"

# Testar parametrização explícita de k
k_custom = 10
custom_box = generate_random_clues(n=n, k=k_custom)
assert len(custom_box.cells) == k_custom

# Verificar se todas as pistas têm coordenadas e valores válidos
for (r, c), val in custom_box.cells.items():
    assert 0 <= r < size and 0 <= c < size, f"Coordenada ({r}, {c})
fora dos limites"
    assert 1 <= val <= size, f"Valor {val} fora do domínio [1, {size}]"

# Testar a matriz gerada pelas pistas
mat = custom_box.to_matrix()
valores_na_matriz = sum(1 for r in range(size) for c in range(size) if
mat[r][c] != 0)
assert valores_na_matriz == k_custom, "A matriz deve conter exatamente
k valores não nulos"

# Testar rejeição de k inválido
try:
    generate_random_clues(n=n, k=-1)
    assert False, "Deveria ter rejeitado k negativo"
except ValueError:
    pass

return "Requisito R4 (Geração de pistas) validado com sucesso!"

test_random_clues()

```

'Requisito R4 (Geração de pistas) validado com sucesso!'

Modelação CSP e escolha do solver (R5)

Porquê o OR-TOOLS CP-SAT?

O enunciado dava liberdade para escolher a tecnologia de resolução. Escolhemos o *CP-SAT* (escolha recomendada pelos professores também) pois este trabalha diretamente com números inteiros (variáveis com domínio de 1 a n^2) e inclui a restrição

`AddAllDifferent`. Esta restrição não faz tentativas cegas, deduz e elimina de imediato os números impossíveis antes de tentar adivinhar.

O que fazer quando o puzzle não tem solução?

Como as pistas são sorteadas à sorte, por vezes calham números que entram em contradição direta, tornando o Sudoku impossível:

- **A nossa decisão:** Quando o solver deteta que não há solução, **retorna imediatamente None** e avisa o utilizador, em vez de ficar num ciclo infinito a tentar sortear pistas novas até dar certo.
- **A razão:** Provar que um problema é impossível é um resultado matemático tão válido e rigoroso como encontrar a solução. Além disso, ficar a sortear pistas em ciclo tornaria o tempo do programa imprevisível.

```
In [ ]: # Implementação do requisito R5 - Modelo e resolução CSP com OR-Tools

class SudokuCSP:
    """Modelo de CSP para uma grelha de Sudoku n^2 x n^2.
    Utiliza o solver CP-SAT da biblioteca OR-Tools.
    Modela cada célula como uma variável inteira com domínio [1, n^2].
    Aplica restrições 'AllDifferent' e valores fixos a grupos genéricos
    (Box).
    """

    def __init__(self, n=3):
        self.n = n
        self.size = n * n
        self.model = cp_model.CpModel()
        self.groups = []

        # Criação das variáveis de decisão: uma por célula no domínio [1,
n^2]
        self.vars = {
            (r, c): self.model.NewIntVar(1, self.size, f"cell_{r}_{c}")
            for r in range(self.size)
            for c in range(self.size)
        }

    def add_groups(self, *groups):
        """Recebe um número arbitrário de grupos (Box, Cube, Path ou
pistas).
        O modelo não distingue a origem ou tipo de cada grupo.
        Para cada grupo:
        - Impõe que todas as suas células tenham valores distintos
        (AllDifferent);
        - Fixa o valor das células que já possuem número atribuído
        (pistas).
        """
        for item in groups:
            if isinstance(item, (list, tuple)):
                for g in item:
                    self._process_group(g)
            else:
                self._process_group(item)

    def _process_group(self, group):
```

```

        """Aplica as restrições de um grupo individual ao modelo CSP."""
        self.groups.append(group)

        # Mapeia as coordenadas das células do grupo para as variáveis do
        # modelo
        group_vars = [self.vars[pos] for pos in group.cells.keys()]

        # Impõe que as células do grupo tenham valores todos diferentes
        if len(group_vars) > 1:
            self.model.AddAllDifferent(group_vars)

        # Fixa os valores das células pré-preenchidas
        for pos, val in group.cells.items():
            if val is not None:
                self.model.Add(self.vars[pos] == val)

    def solve(self):
        """Resolve o CSP montado.
        Retorna a grelha preenchida como matriz n^2 x n^2 de inteiros,
        ou None caso o puzzle não tenha solução (insatisfazível).
        """
        solver = cp_model.CpSolver()
        status = solver.Solve(self.model)

        if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
            grid = [
                [solver.Value(self.vars[(r, c)]) for c in range(self.size)]
                for r in range(self.size)
            ]
            return grid
        return None

```

In []: *# Validação Automática do Requisito R5*

```

def test_sudoku_csp():
    # Teste de resolução consistente para n=2 (grelha 4x4)
    csp = SudokuCSP(n=2)
    b = Box(n=2)
    b.add(0, 0, val=1)
    b.add(0, 1, val=2)
    csp.add_groups(b)

    solucao = csp.solve()
    assert (
        solucao is not None
    ), "O solver deveria encontrar uma solução para um grupo consistente"
    assert (
        solucao[0][0] == 1 and solucao[0][1] == 2
    ), "As pistas fixadas devem ser respeitadas"
    assert (
        len(solucao) == 4 and len(solucao[0]) == 4
    ), "A dimensão da matriz deve ser 4x4 para n=2"

    # Teste de detecção de puzzle impossível (UNSAT)
    # Criamos um grupo com duas células forçadas ao mesmo valor (conflito
    # direto)

```

```

csp_invalido = SudokuCSP(n=2)
b_conflito = Box(n=2)
b_conflito.add(0, 0, val=3)
b_conflito.add(0, 1, val=3)
csp_invalido.add_groups(b_conflito)

solucao_invalida = csp_invalido.solve()
assert (
    solucao_invalida is None
), "O solver tem de sinalizar None quando o puzzle não tem solução"

return "Requisito R5 (Modelo e resolução CSP) validado com sucesso!"

test_sudoku_csp()

```

'Requisito R5 (Modelo e resolução CSP) validado com sucesso!'

Montagem completa

Para fechar o Sudoku completo:

- **Montagem desacoplada (`build_sudoku`)**: Criamos as linhas com `Path`, as colunas com `Path`, os blocos com `Cube` e as pistas com `Box`. Envia tudo para o solver através do método `add_groups`, sem que o solver precise de saber a origem de cada grupo.
- **Validação externa (`validate_solution`)**: Para não confiar cegamente no solver, criamos uma função independente que inspeciona a matriz final número a número. Ela confirma através de conjuntos (`set`) se todas as linhas, colunas e blocos têm números todos de 1 a n^2 sem repetições e se nenhuma das pistas iniciais foi alterada.

In []: *# Implementação do requisito R6 - Montagem completa e visualização*

```

def build_sudoku(n=3, clues= None):
    """Monta a estrutura completa de um Sudoku n^2 x n^2.
    Junta:
    - Todas as linhas (via Path)
    - Todas as colunas (via Path)
    - Todos os blocos n x n (via Cube)
    - Pistas iniciais opcionais (instância de Box)
    Retorna o modelo SudokuCSP configurado com todos os grupos.
    """
    size = n * n
    csp = SudokuCSP(n=n)
    all_groups = []

    # Montar todas as linhas usando troços retos (Path)
    for r in range(size):
        all_groups.append(Path((r, 0), (r, size - 1), n=n))

    # Montar todas as colunas usando troços retos (Path)

```

```

for c in range(size):
    all_groups.append(Path((0, c), (size - 1, c), n=n))

# Montar todos os blocos n x n usando Cube
for bi in range(n):
    for bj in range(n):
        all_groups.append(Cube(bi, bj, n=n))

# Adicionar pistas (se fornecidas)
if clues is not None:
    all_groups.append(clues)

# Injeta todos os grupos no modelo CSP de forma agnóstica
csp.add_groups(all_groups)
return csp

def print_grid(grid, n=3):
    """Apresenta a grelha de Sudoku numa formatação textual legível por
    blocos.
    """
    if grid is None:
        print("Grelha sem solução.")
        return

    size = n * n
    col_width = len(str(size))
    line_sep = "+".join(["-" * (n * (col_width + 1) + 1)] * n)

    for r in range(size):
        if r > 0 and r % n == 0:
            print(f"{line_sep}+")
        row_str = " | "
        for bi in range(n):
            block_cells = [
                str(grid[r][bi * n + c]).rjust(col_width) for c in range(n)
            ]
            row_str += " ".join(block_cells) + " | "
        print(row_str)

```

In []: # Função de verificação formal de soluções

```

def validate_solution(grid, n=3, clues_box=None):
    """Verifica formalmente se uma grelha cumpre todas as regras do
    Sudoku."""
    assert grid is not None, "A grelha a validar não pode ser None."
    size = n * n
    expected_set = set(range(1, size + 1))

    # Validar linhas
    for r in range(size):
        assert (
            set(grid[r]) == expected_set
            ), f"Linha {r} não contém exatamente os valores 1..{size}."

    # Validar colunas
    for c in range(size):

```

```

        col_vals = {grid[r][c] for r in range(size)}
        assert (
            col_vals == expected_set
        ), f"Coluna {c} não contém exatamente os valores 1..{size}."

    # Validar blocos n x n
    for bi in range(n):
        for bj in range(n):
            block_vals = {
                grid[bi * n + r][bj * n + c] for r in range(n) for c in
range(n)
            }
            assert (
                block_vals == expected_set
            ), f"Bloco ({bi}, {bj}) não contém os valores 1..{size}."

    # Validar se as pistas iniciais foram preservadas
    if clues_box is not None:
        for (r, c), val in clues_box.cells.items():
            if val is not None:
                assert (
                    grid[r][c] == val
                ), f"Pista na célula ({r}, {c}) foi alterada de {val} para
{grid[r][c]}."

    return True

```

In []: *# Demonstração e execução completa (n=3 e n=2)*

```

def run_sudoku_demonstration():
    print(" 1. RESOLUÇÃO DE SUDOKU CLÁSSICO (n=3, Grelha 9x9)\n")

    # Gera pistas aleatórias (k razoável para permitir solução fácil, ex:
k=n=3)
    pistas_n3 = generate_random_clues(n=3, k=5)
    csp_n3 = build_sudoku(n=3, clues=pistas_n3)
    solucao_n3 = csp_n3.solve()

    if solucao_n3 is not None:
        print_grid(solucao_n3, n=3)
        validate_solution(solucao_n3, n=3, clues_box=pistas_n3)
        print("\n Solução 9x9 encontrada e validada formalmente!")
    else:
        print("As pistas sorteadas para 9x9 não admitem solução.")
    )

    print("\n 2. DEMONSTRAÇÃO DE GENERALIDADE (n=2, Grelha 4x4)\n")

    pistas_n2 = generate_random_clues(n=2, k=2)
    csp_n2 = build_sudoku(n=2, clues=pistas_n2)
    solucao_n2 = csp_n2.solve()

    if solucao_n2 is not None:
        print_grid(solucao_n2, n=2)
        validate_solution(solucao_n2, n=2, clues_box=pistas_n2)
        print("\n Solução 4x4 encontrada e validada formalmente!")

```

```

else:
    print("As pistas sorteadas para 4x4 não admitem solução.")

    return "Requisito R6 concluído com sucesso!"

run_sudoku_demonstration()

```

1. RESOLUÇÃO DE SUDOKU CLÁSSICO (n=3, Grelha 9x9)

```

| 9 3 8 | 7 6 4 | 1 5 2 |
| 1 2 4 | 8 3 5 | 7 6 9 |
| 5 7 6 | 1 2 9 | 4 8 3 |
+-----+-----+-----+
| 3 9 7 | 5 1 8 | 6 2 4 |
| 6 8 1 | 4 9 2 | 3 7 5 |
| 2 4 5 | 6 7 3 | 8 9 1 |
+-----+-----+-----+
| 4 1 2 | 9 8 7 | 5 3 6 |
| 8 5 9 | 3 4 6 | 2 1 7 |
| 7 6 3 | 2 5 1 | 9 4 8 |

```

Solução 9x9 encontrada e validada formalmente!

2. DEMONSTRAÇÃO DE GENERALIDADE (n=2, Grelha 4x4)

```

| 3 2 | 1 4 |
| 1 4 | 2 3 |
+-----+
| 4 1 | 3 2 |
| 2 3 | 4 1 |

```

Solução 4x4 encontrada e validada formalmente!

'Requisito R6 concluído com sucesso!'

Uso de LLM's

No âmbito deste trabalho prático, recorreu-se pontualmente a ferramentas de Inteligência Artificial (LLM) como suporte de desenvolvimento nas seguintes tarefas:

- **Sintaxe da biblioteca OR-Tools:** Consulta dos nomes dos métodos da biblioteca Google OR-Tools para criar variáveis (`NewIntVar`), aplicar a regra de valores diferentes (`AddAllDifferent`) e obter o resultado final (`solver.value`).
- **Resolução de erros de código (*debugging*):** Apoio na identificação de erros de sintaxe em Python e mensagens de erro do ambiente Marimo.
- **Revisão de texto:** Melhoria da redação das explicações e formatação das fórmulas em Markdown.